Chapter 12 Report

Public Key Cryptography and RSA

Theory, Security, and Real-World Attacks

Course Topic: Chapter 12 — Public Key Cryptography and RSA
Prepared: June 22, 2026

Contents

1. Introduction to Public Key Cryptography

Public key cryptography was introduced to address a fundamental limitation of symmetric encryption: **secure key distribution at scale**. In a symmetric system, two parties must share the same secret key before confidential communication can begin. This requirement is manageable in a small and tightly controlled setting, but it becomes increasingly difficult as the number of communicating participants grows. If a network contains n users and every pair requires a distinct symmetric key, then the total number of shared keys is

$$\frac{n(n-1)}{2}.$$

This quadratic growth creates substantial operational overhead, since each secret key must be generated, transmitted, stored, and periodically replaced through a secure channel.

The **key distribution problem** is not merely an inconvenience; it is a structural obstacle to large-scale secure communication. If the initial exchange of the symmetric key is intercepted or manipulated, then the confidentiality of all subsequent messages is compromised. Consequently, the security of the entire system depends on establishing secret material before encryption can even begin. Public key cryptography offers a different model that reduces this dependency on prior secret sharing.

In the public key setting, each user generates a pair of mathematically related keys: a **public key**, which may be distributed openly, and a **private key**, which must remain secret. Let the public key be denoted by pk and the private key by sk. The general encryption model can be written as

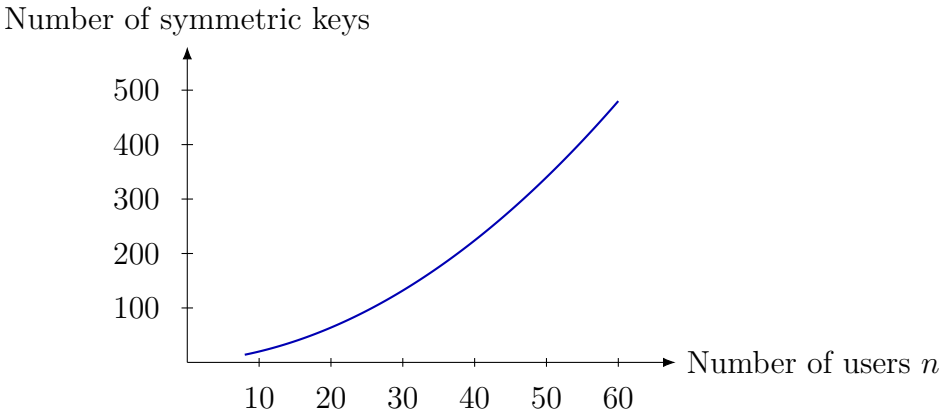
$$c = \mathrm{Enc}_{pk}(m)$$

for the encryption of a message m into a ciphertext c, and

$$m = \mathrm{Dec}_{sk}(c)$$

for decryption using the corresponding private key. The essential idea is that anyone may encrypt a message with the public key, but only the holder of the private key can efficiently recover the plaintext. This separation between an openly shared encryption key and a secret decryption key is the central conceptual advance of public key cryptography.

The significance of this model is twofold. First, it simplifies secure communication between parties who have never met or exchanged a secret in advance. A sender only needs access to the receiver's public key in order to encrypt a message securely. Second, it provides the foundation for **modern secure communication across open networks**. Although practical protocols often combine public key and symmetric techniques for efficiency, the public key framework solves the initial trust and key-establishment problem that symmetric encryption alone cannot resolve conveniently.

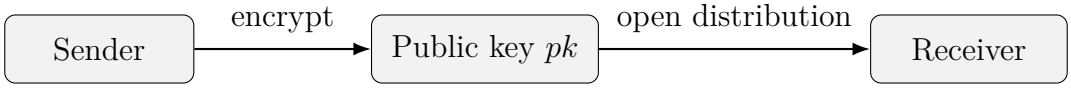


Quadratic growth: in a symmetric-key network with n users, the number of distinct shared keys is

$$\frac{n(n-1)}{2}.$$

As the number of users increases, the system must manage many more secret keys.

Operational overload: each additional key must be generated, exchanged securely, stored, protected, and eventually rotated or replaced.



Public key model: one public key can be distributed openly, while only the private key sk must remain secret.

Figure 1: The symmetric-key model scales poorly because the number of shared secrets grows quadratically, whereas the public key model separates public distribution from private decryption authority.

From a conceptual perspective, public key cryptography changes the question from "How can two parties secretly share the same key?" to "How can one party publish enough information to enable encryption without revealing the ability to decrypt?" This shift leads directly to the study of number-theoretic constructions such as ElGamal and RSA, where security is based on **computational hardness assumptions** rather than on the secrecy of a pre-shared channel. For this reason, public key cryptography is a foundational topic in modern cryptography and an essential starting point for understanding the systems developed in Chapter 12.

2. Mathematical Background

The cryptographic systems discussed in this report rely on a small collection of mathematical ideas from number theory and computational hardness. This section presents only the background necessary to understand ElGamal and RSA. It introduces the concepts that directly support the security arguments and correctness proofs used later.

2.1 Modular Arithmetic

Modular arithmetic studies integers with respect to a fixed modulus $n \geq 2$. For integers a and b, one writes

$$a \equiv b \pmod{n}$$

when n divides $a - b$. In other words, a and b leave the same remainder when divided by n. Computation modulo n allows large values to be reduced into a finite set of residue classes

$$\{0, 1, 2, \ldots, n-1\}.$$

This structure is fundamental in cryptography because encryption and decryption algorithms often involve repeated multiplication and exponentiation modulo a large integer. **Modular arithmetic** is therefore the basic language in which both ElGamal and RSA are expressed.

For example, if $17 \equiv 5 \pmod{12}$, then both integers represent the same residue class modulo 12. Likewise, arithmetic operations are preserved under congruence. If

$$a \equiv b \pmod{n} \quad \text{and} \quad c \equiv d \pmod{n},$$

then

$$a + c \equiv b + d \pmod{n}$$

and

$$ac \equiv bd \pmod{n}.$$

This property makes modular arithmetic especially suitable for algorithmic computation in cryptographic constructions.

2.2 Euler's Totient Function

Euler's totient function, denoted by $\varphi(N)$, counts the number of integers in the set $\{1, 2, \ldots, N-1\}$ that are relatively prime to N. This function plays a central role in RSA. In the important special case where

$$N = pq$$

for two distinct primes p and q, the totient value is

$$\varphi(N) = (p-1)(q-1).$$

The reason is that among the integers from 1 to $N - 1$, the only values not relatively prime to N are those divisible by p or q.

This formula is essential because **RSA key generation** depends on choosing a public

exponent e and a private exponent d such that

$$ed \equiv 1 \pmod{\varphi(N)}.$$

The existence of this modular inverse requires

$$\gcd(e, \varphi(N)) = 1.$$

2.3 Euler's Theorem

Euler's theorem is a basic result connecting modular exponentiation and the totient function. It states that if

$$\gcd(x, N) = 1,$$

then

$$x^{\varphi(N)} \equiv 1 \pmod{N}.$$

This theorem generalizes Fermat's little theorem and provides the **core mathematical justification for RSA decryption**. If the exponents e and d satisfy

$$ed = k\varphi(N) + 1$$

for some integer k, then

$$x^{ed} = x^{k\varphi(N)+1} = \left(x^{\varphi(N)}\right)^k x \equiv x \pmod{N},$$

assuming $\gcd(x, N) = 1$. This is the central idea behind the correctness of the RSA trapdoor permutation.

2.4 Discrete Logarithm Problem

The ElGamal cryptosystem is based on the difficulty of solving the **discrete logarithm problem** in an appropriate cyclic group. Let G be a cyclic group generated by an element g. Given an element

$$h = g^a,$$

the discrete logarithm problem asks for the recovery of the exponent a. Although exponentiation is efficient, the inverse operation is believed to be computationally hard in carefully chosen groups of large order.

This asymmetry between easy forward computation and hard inversion is the same general pattern that appears throughout public key cryptography. In ElGamal, the public key contains a group element of the form g^a, while the private key is the exponent a itself. Security depends on the assumption that an adversary cannot efficiently recover a from the public information alone.

2.5 Computational Diffie–Hellman Assumption

The **Computational Diffie–Hellman (CDH) assumption** formalizes the difficulty of combining exponentiated group elements into a shared secret. Given

$$g, \quad g^a, \quad g^b,$$

the task is to compute

$$g^{ab}.$$

The CDH assumption states that no efficient algorithm can perform this computation with non-negligible probability in the groups used by the scheme.

This problem is closely related to the discrete logarithm problem, but it is not identical to it. Even if an adversary cannot recover a or b individually, the question remains whether the shared value g^{ab} can be computed directly. In many cryptographic settings, CDH is treated as a natural hardness assumption for key agreement and related constructions.

2.6 Decisional Diffie–Hellman Assumption

The **Decisional Diffie–Hellman (DDH) assumption** is stronger than CDH and is particularly important for the security of ElGamal encryption. Given a tuple

$$(g, g^a, g^b, T),$$

the challenge is to decide whether

$$T = g^{ab}$$

or whether T is simply a random group element independent of a and b. Under the DDH assumption, no efficient adversary can distinguish these two cases with significant advantage.

This decisional formulation matters because semantic security requires that ciphertext components appear random to an attacker. In ElGamal, the value $h^r = g^{ar}$ is hidden inside the ciphertext. If the tuple formed from the public values and this hidden term is computationally indistinguishable from random, then the encrypted message is also protected against efficient adversaries.

2.7 Factoring Assumption

RSA is built on a different hardness assumption. Instead of relying on discrete logarithms in groups, it relies on the presumed difficulty of factoring a large composite integer

$$N = pq,$$

where p and q are large primes. The factoring assumption states that, given only N, no efficient algorithm can recover its prime factors when the modulus is chosen sufficiently large and properly generated.

This **factoring assumption** is important because knowledge of p and q immediately reveals

$$\varphi(N) = (p-1)(q-1),$$

which in turn allows the computation of the private exponent d from the public exponent e. Thus, the security of RSA is connected to the difficulty of reconstructing the hidden factorization of N.

Taken together, these mathematical concepts establish the background for the two major systems studied in this chapter. **ElGamal** depends on hardness assumptions related to exponentiation in cyclic groups, especially DDH, whereas **RSA** depends on arithmetic modulo a composite integer and the difficulty of factoring that modulus.

3. ElGamal Cryptosystem

The ElGamal cryptosystem is a public key encryption scheme built from exponentiation in a cyclic group. Its security is tied to the difficulty of distinguishing Diffie–Hellman tuples from random ones. In the presentation used in this chapter, the message is protected by combining a random group-derived pad with the plaintext, so the ciphertext changes each time the same message is encrypted.

3.1 Key Generation

Let G be a cyclic group of prime order q generated by an element g. The receiver chooses a secret exponent

$$a \xleftarrow{\$} \{1, 2, \ldots, q-1\}$$

and computes

$$h = g^a.$$

The public key is

$$pk = (G, g, h),$$

and the secret key is

$$sk = a.$$

The value h is public, but recovering a from g and g^a is believed to be hard in an appropriate group. This is the basic one-way structure underlying ElGamal.

3.2 Encryption

To encrypt a message m, the sender chooses a fresh random exponent

$$r \xleftarrow{\$} \{1, 2, \ldots, q-1\}.$$

The ciphertext is then formed as

$$C = (u, v) = (g^r, h^r \cdot m).$$

The first component $u = g^r$ reveals the randomizer in exponentiated form, while the second component masks the message by multiplying it with the shared term $h^r = g^{ar}$. The crucial feature is that the value r is newly sampled for every encryption, so even identical plaintexts produce different ciphertexts.

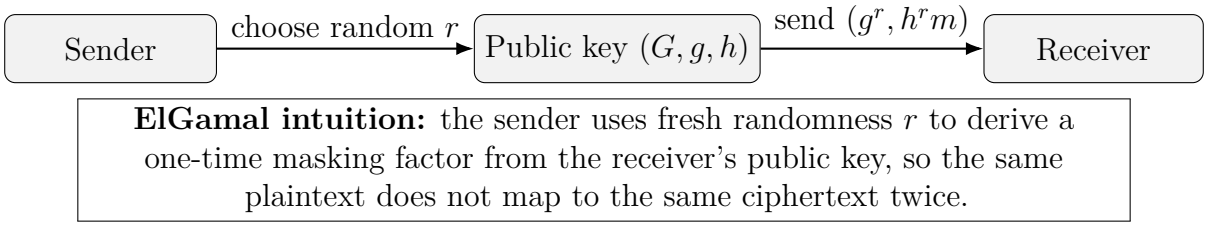


Figure 2: High-level ElGamal encryption flow.

3.3 Decryption

Given a ciphertext

$$(u, v) = (g^r, h^r \cdot m),$$

the receiver uses the secret key a to compute

$$u^a = (g^r)^a = g^{ra} = h^r.$$

The message is then recovered by removing the mask:

$$m = \frac{v}{u^a}.$$

Since

$$\frac{h^r \cdot m}{(g^r)^a} = \frac{g^{ar} \cdot m}{g^{ar}} = m,$$

decryption is correct.

3.4 Security Analysis

The important security claim is that ElGamal is **semantically secure** under the **Decisional Diffie–Hellman assumption**. The structure of the ciphertext is

$$(g^r, h^r \cdot m) = (g^r, g^{ar} \cdot m).$$

If an adversary could distinguish encryptions of two chosen messages, then that adversary would be able to determine whether a tuple of the form

$$(g, g^a, g^r, T)$$

contains

$$T = g^{ar}$$

or instead contains a random group element. This gives the reduction idea:

$$\text{Breaking ElGamal semantic security} \Longrightarrow \text{distinguishing a DDH tuple.}$$

The intuition is straightforward. If $T = g^{ar}$, then the challenge ciphertext is a valid ElGamal encryption. If T is random, then the masking term behaves like an independent one-time pad in the group setting, so the adversary gains no useful information about the message. Therefore, any non-negligible success in distinguishing messages would contradict DDH hardness.

3.5 Small Worked Example

For illustration, consider a small example that is mathematically easy to verify by hand. Let the group be the multiplicative group modulo 23, and choose generator

$$g = 5.$$

Suppose the receiver chooses

$$a = 6.$$

Then

$$h = g^a = 5^6 \equiv 8 \pmod{23},$$

so the public key is $(23, 5, 8)$ and the secret key is 6.

Now let the message be

$$m = 10,$$

and let the sender choose random

$$r = 7.$$

The sender computes

$$u = g^r = 5^7 \equiv 17 \pmod{23}$$

and

$$h^r = 8^7 \equiv 12 \pmod{23}.$$

Thus the second ciphertext component is

$$v = h^r \cdot m \equiv 12 \cdot 10 \equiv 5 \pmod{23}.$$

The ciphertext is therefore

$$C = (17, 5).$$

To decrypt, the receiver computes

$$u^a = 17^6 \equiv 12 \pmod{23}.$$

The inverse of 12 modulo 23 is 2, because

$$12 \cdot 2 \equiv 1 \pmod{23}.$$

Hence

$$m = v \cdot (u^a)^{-1} \equiv 5 \cdot 2 \equiv 10 \pmod{23},$$

which recovers the original message.

This example is intentionally small and insecure in practice, but it clearly demonstrates the three core stages of ElGamal: **key generation**, **randomized encryption**, and **mask removal through exponentiation with the secret key**.

4. RSA Trapdoor Permutation

RSA is most naturally understood as a **trapdoor permutation** on the multiplicative group modulo a composite integer. The public key defines a function that is easy to evaluate, while the private key provides the hidden information required to invert it efficiently.

4.1 Key Generation

RSA begins by selecting two large distinct primes

$$p \quad \text{and} \quad q,$$

and forming the modulus

$$N = pq.$$

The Euler totient of N is

$$\varphi(N) = (p-1)(q-1).$$

Next, one chooses a public exponent e satisfying

$$\gcd(e, \varphi(N)) = 1.$$

Because e is relatively prime to $\varphi(N)$, it has a multiplicative inverse modulo $\varphi(N)$. The private exponent d is therefore defined by

$$ed \equiv 1 \pmod{\varphi(N)}.$$

Equivalently, there exists an integer k such that

$$ed = k\varphi(N) + 1.$$

The public key is

$$pk = (N, e),$$

and the private key is

$$sk = d.$$

The factorization of N is not published. This hidden factorization is what allows the private exponent to be computed in the first place.

4.2 Encryption

For a message M in the multiplicative group $\mathbb{Z}_N^*$, RSA encryption is defined by

$$C = M^e \bmod N.$$

This is efficient even when N is very large because modular exponentiation can be performed using repeated squaring.

4.3 Decryption

Decryption uses the private exponent d:

$$M = C^d \bmod N.$$

Since $C = M^e$, decryption computes

$$C^d = (M^e)^d = M^{ed}.$$

The correctness of RSA is therefore equivalent to showing that

$$M^{ed} \equiv M \pmod{N}.$$

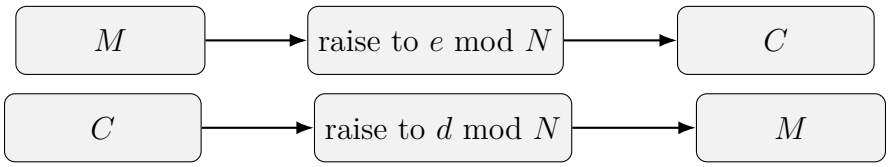


Figure 3: RSA applies the public exponent in the forward direction and the private exponent in the inverse direction.

4.4 Trapdoor Permutation Explanation

A **trapdoor permutation** is a bijection that is easy to compute but hard to invert without special information. RSA fits this pattern. The map

$$M \mapsto M^e \bmod N$$

is public and efficiently computable. However, inverting it amounts to extracting an eth root modulo N, which is believed to be computationally hard without the trapdoor information.

The trapdoor is the secret exponent d, or equivalently the factorization of N. Once the factorization is known, one can compute $\varphi(N)$ and thereby determine d. Without that hidden structure, inversion is presumed to be infeasible for sufficiently large parameters.

4.5 Correctness Proof

The correctness proof follows directly from Euler's theorem. Since

$$ed = k\varphi(N) + 1,$$

we have

$$M^{ed} = M^{k\varphi(N)+1} = \left(M^{\varphi(N)}\right)^k \cdot M.$$

If $M \in \mathbb{Z}_N^*$, then $\gcd(M, N) = 1$, so Euler's theorem implies

$$M^{\varphi(N)} \equiv 1 \pmod{N}.$$

Substituting this into the previous expression gives

$$M^{ed} \equiv 1^k \cdot M \equiv M \pmod{N}.$$

Therefore,

$$C^d \equiv (M^e)^d \equiv M \pmod{N},$$

which proves that decryption correctly recovers the original message.

This proof is important because it shows that RSA is not merely an algorithmic trick. Its correctness is rooted in number-theoretic structure, specifically the relationship between the exponents e and d modulo $\varphi(N)$.

5. Why Textbook RSA is Insecure

Although RSA provides a mathematically elegant trapdoor permutation, **textbook RSA is not a secure encryption scheme**. The function

$$E(M) = M^e \bmod N$$

is deterministic and algebraically structured. These properties make it vulnerable to several attacks that violate semantic security and permit direct ciphertext manipulation.

5.1 Deterministic Encryption

In a secure encryption scheme, encrypting the same message twice should not reveal that the plaintexts are identical. Textbook RSA fails this requirement because it is deterministic:

$$E(M) = M^e \bmod N$$

has exactly one output for each message M. Therefore, if an adversary sees two ciphertexts

$$C_1 = E(M) \quad \text{and} \quad C_2 = E(M),$$

then

$$C_1 = C_2.$$

This immediately leaks equality information. An attacker can also perform a dictionary attack against predictable messages by encrypting candidate plaintexts under the public key and comparing the result with the observed ciphertext. For this reason alone, textbook RSA is **not semantically secure**.

5.2 Small Message Attack

Another weakness appears when the plaintext is numerically small. Suppose

$$M^e < N.$$

Then the modular reduction does not actually change the value, and the ciphertext is simply

$$C = M^e$$

over the ordinary integers. In that case, an attacker can recover the message by taking the usual integer eth root:

$$M = \sqrt[e]{C}.$$

This attack is especially dangerous when the public exponent e is small and the plaintext space contains short or structured messages. It shows that arithmetic correctness does not imply cryptographic secrecy.

5.3 Multiplicative Property Attack

Textbook RSA also preserves multiplication:

$$E(M_1) \cdot E(M_2) \equiv M_1^e M_2^e \equiv (M_1 M_2)^e \equiv E(M_1 M_2) \pmod{N}.$$

This **multiplicative property** means that ciphertexts can be modified in a meaningful way without knowing the underlying plaintexts. If an attacker holds a ciphertext $C = E(M)$ and chooses some value t, then

$$C' = C \cdot E(t) \equiv E(Mt) \pmod{N}$$

is a valid ciphertext for the related plaintext Mt.

This violates the intuition that encrypted data should resist algebraic tampering. Even if the attacker does not fully recover the message, the ability to transform ciphertexts into predictable related ciphertexts is already a serious failure of security.

5.4 Lecture Attack Example

The lecture example illustrates how a short encrypted session key can be attacked much faster than brute force. Suppose a browser encrypts a 64-bit session key K using textbook RSA:

$$C = K^e \bmod N.$$

Now suppose that

$$K = K_1 K_2$$

with

$$K_1, K_2 < 2^{34}.$$

Then

$$C \equiv K_1^e K_2^e \pmod{N}.$$

An attacker can use a meet-in-the-middle strategy:

1. Precompute a table of values derived from one side, such as

$$\frac{C}{K_1^e} \pmod N$$

   for all candidate values of K_1.

2. Compute

$$K_2^e \pmod N$$

   for all candidate values of K_2 and check for matches in the table.

This reduces the work dramatically compared with exhaustive search over all 2^{64} possibilities. Instead of searching the full key space directly, the attacker exploits the multiplicative structure of RSA to divide the problem into two smaller searches.

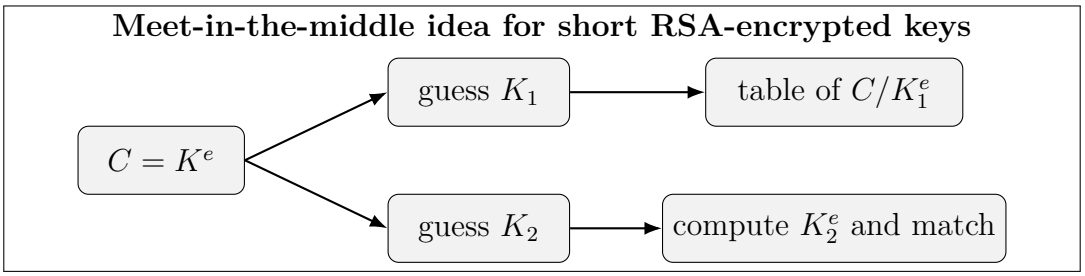


Figure 4: A structured plaintext space can make textbook RSA vulnerable to faster-than-brute-force search.

These attacks illustrate the central lesson of this section: **RSA as a trapdoor permutation is not automatically a secure public key encryption scheme**. Additional preprocessing is required to add randomness and remove exploitable algebraic structure.

6. RSA Padding and Secure RSA Encryption

The weaknesses of textbook RSA arise because the raw function

$$M \mapsto M^e \bmod N$$

is deterministic and algebraically structured. To convert RSA into a practical encryption scheme, one must first transform the message using a randomized encoding process. This transformation is called **padding**.

6.1 Why Padding Is Needed

Padding serves two essential purposes. First, it introduces **randomness**, ensuring that the same message does not always encrypt to the same ciphertext. Second, it adds

structure, so that malformed or manipulated inputs can be recognized and rejected after decryption.

Conceptually, padding changes the encryption process from

$$C = M^e \bmod N$$

to

$$C = \mathrm{RSA}\Big(\mathrm{Pad}(M; r)\Big) = \mathrm{Pad}(M; r)^e \bmod N,$$

where r is fresh randomness drawn independently for every encryption. Because the RSA permutation is now applied to a randomized encoding of the message rather than to the message itself, the determinism and the multiplicative structure exploited against textbook RSA no longer translate into useful attacks: equality tests, dictionary attacks, small-message root extraction, and chosen-ciphertext manipulation all become much harder.

A useful way to state the design goal is that padding should turn RSA into a scheme that is **semantically secure**, and ideally secure against **adaptive chosen-ciphertext attacks** (IND-CCA2), so that an adversary learns nothing from a ciphertext even when allowed to query a decryption oracle on other ciphertexts.

6.2 PKCS#1 v1.5

One historically important padding scheme is **PKCS#1 v1.5**. Let k be the byte length of the modulus N. Before RSA encryption, a message M of mLen bytes is embedded into a k-byte encoded block of the form

$$\underbrace{\texttt{00}}_{1} \,\|\, \underbrace{\texttt{02}}_{1} \,\|\, \underbrace{\mathrm{PS}}_{\geq 8} \,\|\, \underbrace{\texttt{00}}_{1} \,\|\, \underbrace{M}_{\mathrm{mLen}}.$$

The components have specific roles:

- the leading `00` guarantees that the block, read as an integer, is smaller than N;
- the block type byte `02` signals that the block is intended for *encryption*;
- the padding string PS consists of **nonzero** random bytes and must be at least 8 bytes long, which is what supplies the randomness;
- the `00` separator marks the end of the padding and the start of the message.

Because PS must occupy at least 8 bytes and the fixed bytes occupy 3 more, the message length is bounded by $\mathrm{mLen} \leq k - 11$. To decrypt, one computes $C^d \bmod N$, re-expresses it as a k-byte block, and checks that it has the expected `00 02` prefix, a nonzero padding region, and a `00` separator; the message is whatever follows.

This design was widely deployed and was a major improvement over textbook RSA. However, it was later shown to be vulnerable to adaptive chosen-ciphertext attacks whenever an implementation revealed whether a decrypted block had valid PKCS#1 formatting (see Section 7). PKCS#1 v1.5 therefore illustrates a recurring theme in cryptography: an encoding rule can look strong on paper yet still fail when combined with an implementation that leaks even one bit about the decrypted value.

6.3 OAEP

A more modern and principled design is **Optimal Asymmetric Encryption Padding (OAEP)**, introduced by Bellare and Rogaway. OAEP preprocesses the message using fresh randomness together with two hash-based mask generation functions, denoted G and H, before the RSA exponentiation is applied. At a high level it:

- adds a fresh random seed to every encryption,
- mixes the message and the seed through a two-round Feistel-like network built from G and H, and
- produces a structured RSA input in which every output bit depends on the entire message and seed.

Concretely, let hLen be the output length of the hash and k the byte length of N. The message is first formatted into a data block

$$\mathrm{DB} = \mathrm{lHash} \,\|\, \mathrm{PS} \,\|\, \mathtt{01} \,\|\, M,$$

where lHash is the hash of an optional label and PS is a run of zero bytes. A random seed of length hLen is then drawn, and the encoded message is computed as

$$\begin{aligned} \mathrm{maskedDB} &= \mathrm{DB} \oplus G(\mathrm{seed}), \\ \mathrm{maskedSeed} &= \mathrm{seed} \oplus H(\mathrm{maskedDB}), \\ \mathrm{EM} &= \mathtt{00} \,\|\, \mathrm{maskedSeed} \,\|\, \mathrm{maskedDB}, \end{aligned}$$

and EM is the integer fed into the RSA permutation. Decryption simply reverses the two rounds: H recovers the seed from maskedSeed and maskedDB, then G recovers DB, after which the formatting of DB is verified.

OAEP is important because it comes with stronger theoretical guarantees. The construction has the *all-or-nothing* property: flipping a single bit of EM scrambles the recovered seed and data block unpredictably, so an attacker cannot transform a valid ciphertext into another valid one. In the random oracle model, RSA-OAEP is provably **IND-CCA2 secure** under the RSA assumption, which is precisely the resistance to adaptive chosen-ciphertext attacks that PKCS#1 v1.5 lacked.

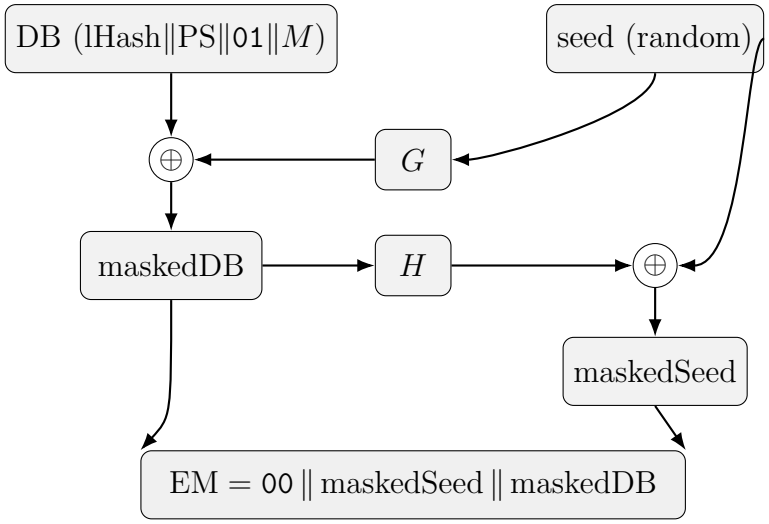


Figure 5: OAEP combines the data block DB and a random seed through two mask generation functions G and H before RSA is applied. Each output bit of EM depends on the whole message and the whole seed.

The broader lesson is that **secure RSA encryption is not just "RSA plus a message"**. It is the RSA permutation composed with a carefully designed randomized encoding procedure. Without that preprocessing step, the trapdoor permutation remains mathematically elegant but cryptographically unsafe.

7. Real-World Attack Case Study: Bleichenbacher Attack

Among the most influential practical attacks on RSA encryption is the **Bleichenbacher attack**, published by Daniel Bleichenbacher in 1998. It targeted implementations of RSA with PKCS#1 v1.5 padding—including the RSA key-exchange used in SSL/TLS—and demonstrated that even a mathematically strong primitive can fail when an implementation leaks a single bit of information about decrypted data.

7.1 Background

PKCS#1 v1.5 was widely used in early web and protocol implementations. As described in Section 6, a valid decrypted block for encryption must begin with the bytes `00 02`, followed by at least eight nonzero padding bytes, a `00` separator, and the message. Servers typically checked whether the decrypted block satisfied this format before continuing the protocol. If the padding was invalid, the implementation often terminated with an error or otherwise behaved observably differently from the valid case.

This difference in behavior created the opening for Bleichenbacher's attack. The attacker does not need the private key, nor does the attacker need to break RSA directly. Instead, the attacker exploits the server's reaction to carefully modified ciphertexts.

7.2 The Padding Oracle

The attack relies on a **padding oracle**: any mechanism that tells the attacker whether a decrypted plaintext satisfies the required PKCS#1 structure. For example,

- valid padding → the server continues processing (or responds slowly, or returns a different error),
- invalid padding → the server returns an error or behaves differently.

Even this single bit of information is enough to be dangerous. Because a conforming block starts with `00 02`, "padding is valid" tells the attacker that the decrypted integer lies in a narrow, predictable range, and each query refines what the attacker knows about the hidden plaintext.

7.3 Attack Flow

Let k be the byte length of N and define

$$B = 2^{8(k-2)}.$$

A plaintext m whose encoding begins with the bytes `00 02` satisfies, as an integer,

$$2B \leq m \leq 3B - 1.$$

This interval is exactly what a positive oracle answer reveals.

Suppose the attacker intercepts a target ciphertext

$$C = M^e \bmod N.$$

The attacker chooses a multiplier s and forms a modified ciphertext

$$C' = C \cdot s^e \bmod N.$$

By the multiplicative property of RSA, when the server decrypts C' it obtains

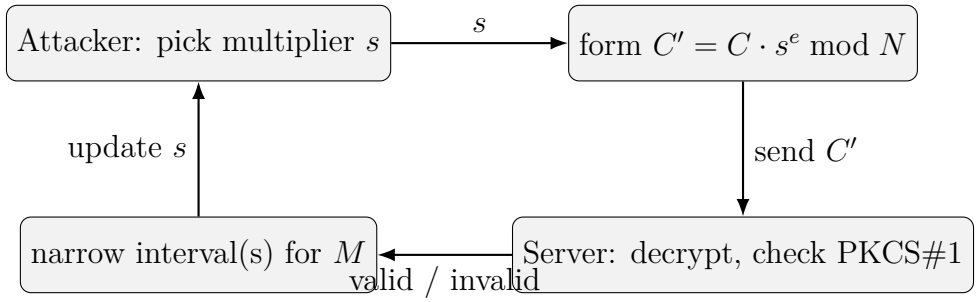
$$M' = Ms \bmod N.$$

The oracle then reveals whether M' is PKCS#1 conforming, i.e. whether

$$2B \leq Ms - rN \leq 3B - 1 \quad \text{for some integer } r.$$

Each conforming value of s yields a linear constraint of this form and therefore pins M down to a union of subintervals of $[2B, 3B - 1]$. By searching for successive multipliers

s in a structured way and intersecting the resulting constraints, the attacker repeatedly halves the set of candidate values until a single interval of width one remains; its endpoint is the original plaintext M. The attack is therefore not a direct algebraic inversion of RSA but an **adaptive interval-narrowing search** driven by oracle feedback, typically requiring on the order of tens of thousands to a few million queries depending on the variant.



Core idea: a "valid" answer means the hidden value $Ms \bmod N$ lies in the PKCS#1 interval $[2B, 3B - 1]$. Looping over chosen multipliers s and intersecting these constraints shrinks the candidate set for M until only the true plaintext remains—turning a one-bit validity check into a full decryption oracle.

Figure 6: Bleichenbacher's attack is an adaptive feedback loop: each oracle bit narrows the interval that must contain the plaintext.

7.4 Why This Matters

The Bleichenbacher attack changed how practitioners think about encryption security. The RSA function itself was not broken. Instead, the failure came from the interaction between:

- structured padding that defines a recognizable "valid" format,
- an implementation that checks that format, and
- an externally visible difference in behavior between the valid and invalid cases.

This is a powerful lesson. A system may be built from strong mathematical components yet still become insecure if its implementation leaks partial information. The attack also proved remarkably durable: nearly two decades later, variants such as **DROWN** (2016, exploiting legacy SSLv2) and **ROBOT** — "Return Of Bleichenbacher's Oracle Threat" (2017) — showed that the same oracle persisted in widely deployed TLS stacks, because PKCS#1 v1.5 is hard to implement without some timing or error-message side channel.

For this reason, Bleichenbacher's attack is far more than a historical curiosity. It is the classic example of how **implementation-level leakage can completely defeat a**

mathematically sound primitive, and it strongly motivated the move toward provably CCA-secure encodings such as OAEP and, more broadly, toward constant-time, leakage-resistant implementations.

8. Conclusion

Public key cryptography solves a problem that symmetric encryption alone handles poorly: secure key establishment between parties that do not already share a secret. This chapter examined two major constructions that achieve this goal through different mathematical foundations.

The first construction, **ElGamal**, is based on exponentiation in a cyclic group and derives its security from the hardness of Diffie–Hellman-style problems, especially the **Decisional Diffie–Hellman assumption**. Its use of fresh randomness during encryption is essential: that randomness is what prevents ciphertexts from revealing message equality and is what makes semantic security possible.

The second construction, **RSA**, is built from a trapdoor permutation on arithmetic modulo a composite integer. Its correctness follows from Euler's theorem, while its practical security is tied to the hardness of factoring the modulus. RSA shows how elegant number-theoretic structure can yield an efficient public key primitive with a clear forward direction and a hidden inverse.

At the same time, the chapter showed that **textbook RSA is insecure**. Deterministic encryption, vulnerability to small-message attacks, and the multiplicative structure of the raw function all prevent it from serving as a secure encryption scheme on its own. A trapdoor permutation is therefore *not* the same thing as secure public key encryption.

To obtain practical security, RSA must be combined with **padding**. Padding introduces randomness and structured preprocessing before exponentiation, removing the most obvious weaknesses of the textbook scheme. The older **PKCS#1 v1.5** encoding improved matters significantly, but history showed that it could still fail under adaptive chosen-ciphertext conditions. More modern designs such as **OAEP** were introduced to close that gap, offering provable IND-CCA2 security in the random oracle model.

The **Bleichenbacher attack** is the clearest real-world illustration of this principle, and its long-lived descendants (DROWN, ROBOT) show that the danger was not a one-time bug but a structural pitfall. The attack undermined deployed systems not by breaking the underlying mathematics, but by exploiting a single bit of information leaked during decryption. In that sense, the chapter's most important lesson is broader than ElGamal or RSA alone: **cryptographic security depends on both theory and implementation**. Strong hardness assumptions are necessary, but they are not sufficient unless the complete scheme—encoding, protocol, and implementation—is designed and deployed with equal care.